# MAS508-G Control Theory
# Lecture 6: Advanced Control

Prof. Jing Zhou

University of Agder

Department of Engineering Sciences

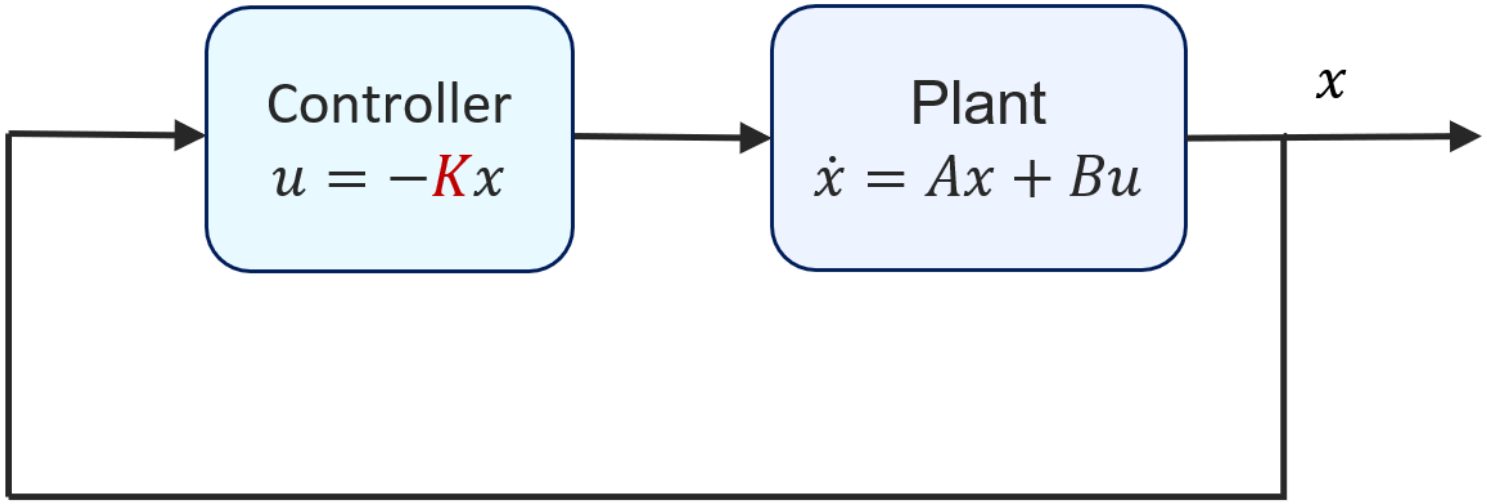
Email: jing.zhou@uia.no

# Model Predictive Control (MPC)

https://en.wikipedia.org/wiki/Model_predictive_control
https://laurentlessard.com/teaching/me7247/lectures/lecture%2020%20-%20model%20predictive%20control.pdf

# Linear Quadratic Regulator - LQR

- Linear Quadratic Regulator (LQR) is an optimization method for
- It is used to **find the gain $K$ of a state-feedback control.**




- **Minimize:** $J = \int_0^\infty x(t)^T Qx(t) + u(t)^T Ru(t)dt$

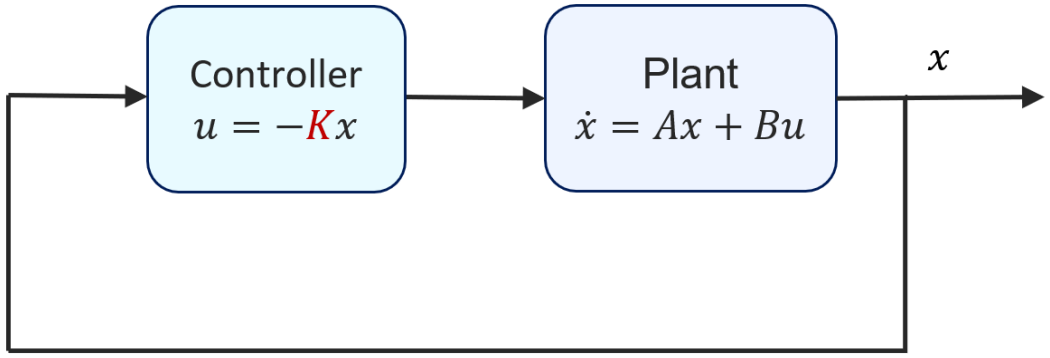
  Subject to $\dot{x} = Ax + Bu$

# LQR

- Linear Quadratic Regulator (LQR) is an optimization method.
- It is used to **find the gain $\boldsymbol{K}$** of a **state feedback control $\boldsymbol{u = -Kx}$**.

- Minimize $J = \int_0^\infty x(t)^T Qx(t) + u(t)^T Ru(t)dt$

- Find $P$ that solves the Algebraic Riccati Equation. $A^T P + PA + Q - PBR^{-1}B^T P = 0$
- Find the optimal control gain $K$. $K = R^{-1}B^T P$



# MPC

- Model predictive Control (MPC) is an advanced method of process control that is used to control a process while satisfying a set of constraints.

Main Advantages

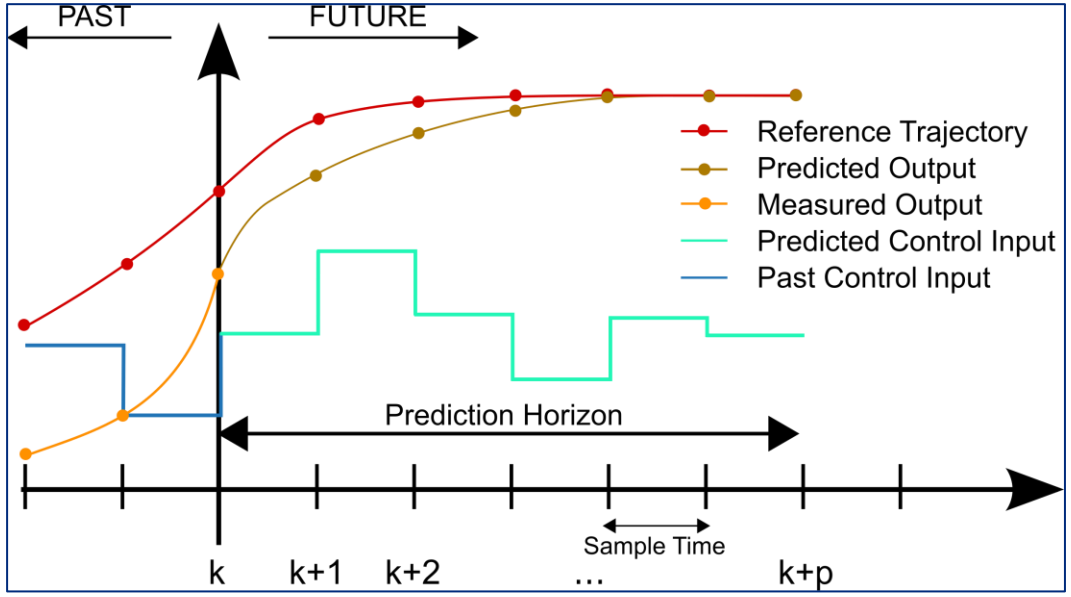
- It allows the current timeslot to be optimized, while keeping future timeslots in account.
- MPC can anticipate future events and can take control actions accordingly.
- Can easily handle nonlinear and time-varying plant dynamics

# Concept - MPC

- MPC is an advanced method of process control that is used to control a process while satisfying a set of constraints.
- Control algorithms based on
  - Numerically solving an optimization problem at each step
  - Constrained optimization – typically Quadratic Problem (QP)
  - Receding horizon control (RHC)
    - ✓ At each time step, compute control by solving an open-loop optimization problem for the prediction horizon
    - ✓ Apply the first value of the computed control sequence
    - ✓ At the next time step, get the system state and re-compute

# Theory - MPC

- MPC is based on iterative, finite-horizon optimization of a plant model.
- At time $t$ the current plant state is sampled and a cost minimizing control strategy is computed (via a numerical minimization algorithm) for a relatively short time horizon in the future: $[t, t+T]$.
- Repeated optimization: Only the first step of the control strategy is implemented, then the plant state is sampled again, and the calculations are repeated starting from the new current state, yielding a new control and new predicted state path.
- The prediction horizon keeps being shifted forward and for this reason MPC is also called receding horizon control.




https://en.wikipedia.org/wiki/Model_predictive_control

# Principle - MPC

MPC is a multivariable control algorithm that uses:

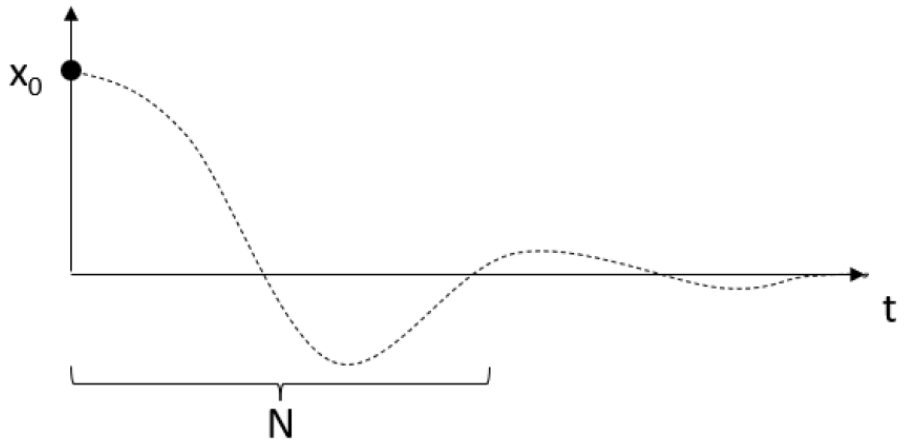
- an internal dynamic model of the process
- a cost function $J$ over the receding horizon
- an optimization algorithm minimizing the cost function using the control input.

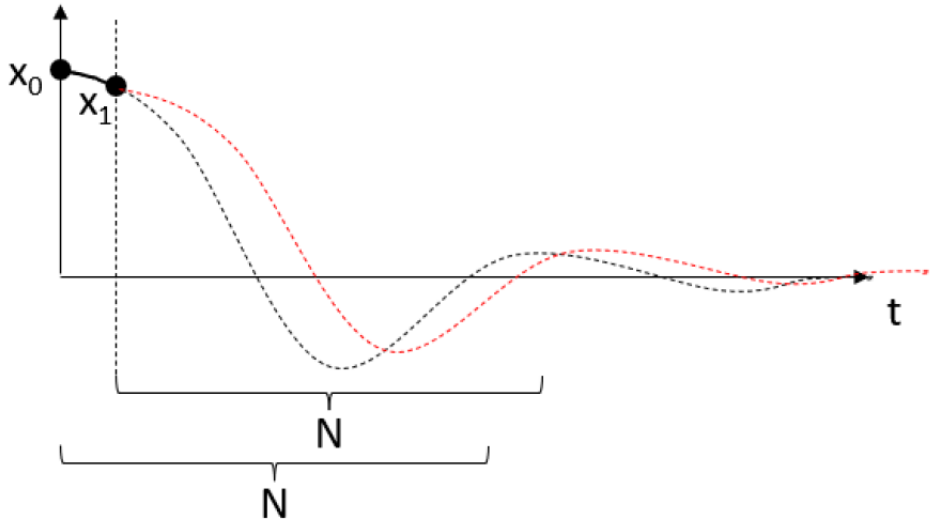
*Example of a quadratic cost function for optimization:*

$$\begin{aligned} \hat{J}(z) \quad = \quad & \underset{\substack{u_0,u_1,\ldots,u_{N-1} \\ x_1,x_2,\ldots,x_N}}{\text{minimize}} \quad \sum_{k=0}^{N-1} \left( x_k^\mathsf{T} Q x_k + u_k^\mathsf{T} R u_k \right) \\ & \text{s.t.} \quad x_0 = z \\ & \qquad x_{k+1} = A x_k + B u_k \quad \text{for } k = 0, 1, \ldots \\ & \qquad x_k \in X, \quad u_k \in U \end{aligned}$$

# Design of MPC

1. Choose a finite horizon N.
2. Solve the finite horizon LQR problem using $(x_0, N)$. This produces a set of optimal decisions $\{u_0^*, u_1^*, \dots, u_{N-1}^*\}$ and corresponding predicted states$\{x_1^*, x_2^*, \dots, x_N^*\}$.
3. Apply the first action $u_0^*$ to move to $x_1$. (Although we expect to move to $x_1^*$, there may be some noise, unmodeled dynamics, etc. So, we may not end up exactly at $x_1^*$.)
4. Solve the finite horizon LQR problem using $(x_1, N)$.
5. Apply the first action again, to move to $x_2$.
6. Repeat.



# MPC vs. LQR

MPC and LQR are both optimal control, with different schemes of setting up optimization costs.

- MPC looks at fixed length, often gradually weighted sets of error functions.
- LQR looks at all linear system inputs and provides the system that will reduce the total error across the frequency spectrum, trading off state error against input frequency.
- LQR has better global stability properties, but MPC often has more locally optimal and complex performance.

Differences between MPC and LQR

- LQR optimizes across the entire time window.
- MPC optimizes in a receding time window (finite horizon).
- LQR solves the optimization problem for whole time horizon.
- MPC solves the optimization problem in a smaller fixed time window and may obtain a suboptimal solution.

MPC: nonlinear systems

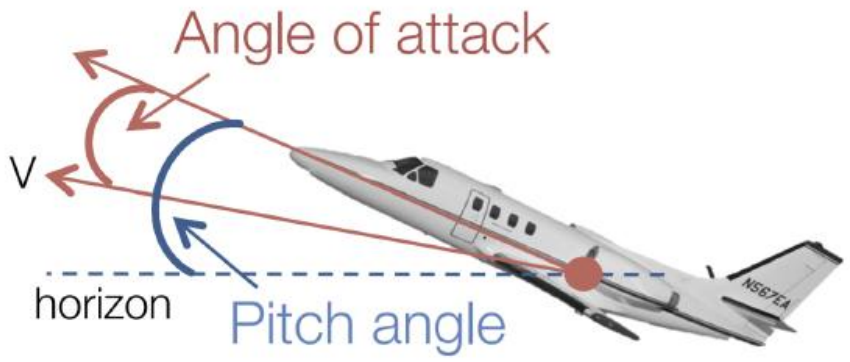
# Advantages - MPC

- Straightforward formulation, based on well understood concepts
- Handle constraints
- Explicit use of a model
- Well understood tuning parameters
  - Prediction horizon
  - Optimization problem setup
- Can easily handle nonlinear and time-varying plant dynamics
- Easier to maintain: changing model or specifications does not require complete redesign.

# Example 1: Pitch-altitude control

LTI model with at an altitude of 5000 m and a speed of 128.2 m/sec.

$$\dot{x} = \begin{bmatrix} -1.2822 & 0 & 0.98 & 0 \\ 0 & 0 & 1 & 0 \\ -5.4293 & 0 & -1.8366 & 0 \\ -128.2 & 128.2 & 0 & 0 \end{bmatrix} x + \begin{bmatrix} -0.3 \\ 0 \\ -17 \\ 0 \end{bmatrix} u$$




- $x_1$: angle of attack (in radians).
- $x_2$ : pitch angle (in radians).
- $x_3$ : pitch rate (in rad/sec).
- $x_4$ : altitude (in meters).
- $u$: elevator angle (in radians). The elevator is a control surface on the horizontal tail that can be angled down (positive angle) or angled up (negative angle).

The goal is to regulate the altitude to zero (correspond to the nominal altitude of 5000 m) subject to the constraints:

- Sampling rate of $T_s = 0.25$ seconds.
- Constraints:
  - $|\boldsymbol{u}| \leq \mathbf{0.262}\ (\pm \mathbf{15^o})$
  - $|\dot{\boldsymbol{u}}| \leq \mathbf{0.524}\ (\pm \mathbf{30^o/sec})$
  - $|\boldsymbol{x_2}| \leq \mathbf{0.349}\ (\pm \mathbf{20^o})$

https://laurentlessard.com/teaching/me7247/lectures/lecture%2020%20-%20model%20predictive%20control.pdf

# Example 1

Design a constrained LQR. The optimization problem:

$$\underset{u,x}{\text{minimize}} \quad \sum_{t=0}^{\infty} x_t^\mathsf{T} Q x_t + u_t^\mathsf{T} R u_t \qquad \text{(state update)}$$

$$\text{subject to:} \quad x_{t+1} = A x_t + B u_t$$

$$\begin{bmatrix} -\infty \\ -0.349 \\ -\infty \\ -\infty \end{bmatrix} \leq x_t \leq \begin{bmatrix} \infty \\ 0.349 \\ \infty \\ \infty \end{bmatrix} \qquad \text{(pitch constraint)}$$

$$-0.262 \leq u_t \leq 0.262 \qquad \text{(elevator constraint)}$$

$$-0.524 \leq \frac{u_{t+1} - u_t}{T_s} \leq 0.524 \qquad \text{(elevator rate constraint)}$$

# Example 1- LQR no constraints

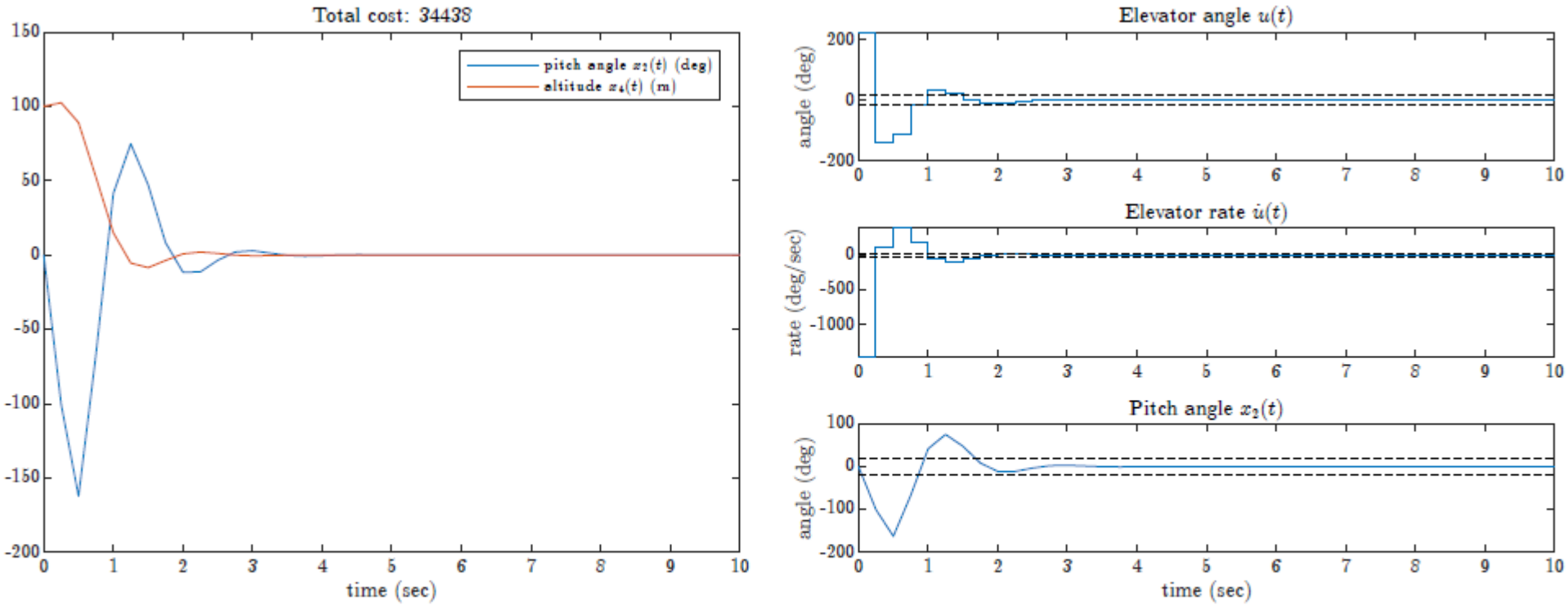



Figure Ordinary LQR with $Q = I$ and $R = 100$. No constraints are imposed and naturally, the solution does not obey any of the constraints.

# Example 1: LQR with constraint on $u$

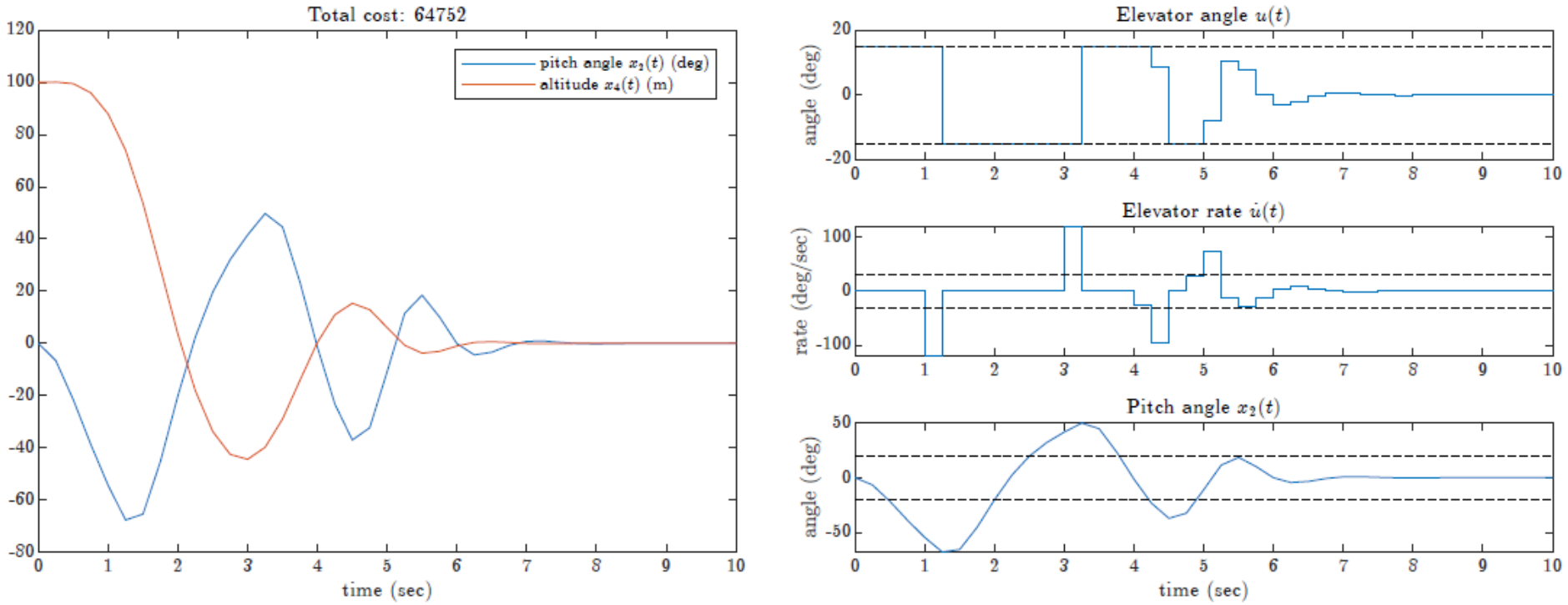



Figure Ordinary LQR with $Q = I$ and $R = 100$, except this time, we clip the inputs so if $u_t > 0.262$, we set $u_t = 0.262$. Similarly, if $u_t < -0.262$, we set $u_t = -0.262$. This forces $u_t$ to satisfy the constraints, but we do not satisfy the other two constraints. The clipping also induces some unwanted oscillation in the response.

# Example 1: MPC with $N = 10$

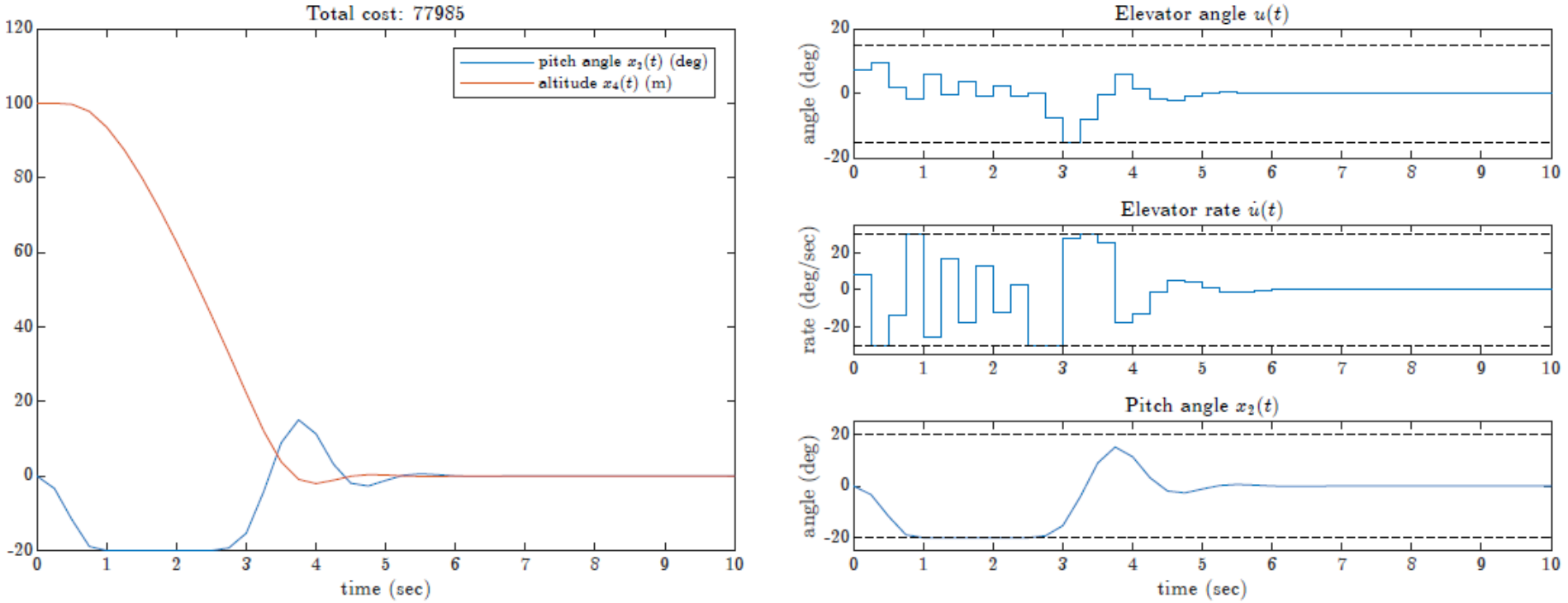



Figure . MPC with $Q = I$ and $R = 100$ and a horizon of $N = 10$, using all constraints. All constraints are satisfied, and the solution looks pretty good!

# Example 1: MPC with $N = 3$

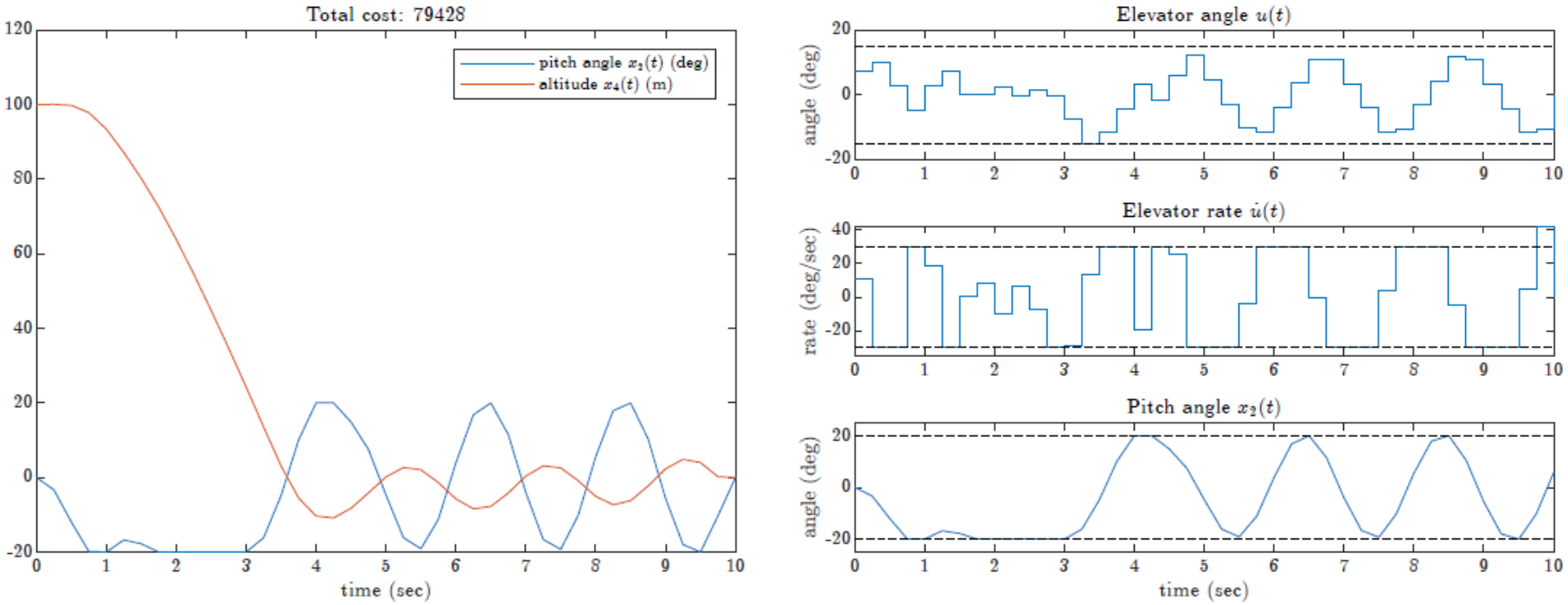



Figure MPC with $Q = I$ and $R = 100$ and a horizon of $N = 3$, using all constraints. The horizon is too short: a feasible solution is found, but it is unstable and oscillates endlessly, so the infinite-horizon cost will be infinite. Note: if we use $N = 2$ we lose feasibility on the third timestep.

# Summary of MPC

- Model predictive control (MPC)
  - MPC is an optimal control.
  - MPC can handle state and control constraints .
  - MPC optimizes in a fixed length (finite horizon time window) at every time step of system.
  - MPC often has more locally optimal.
  - Predictive control computed via quadratic programming.

- System model: Many parameters and their influence on the control
  - System model, weights, horizons, constraints, …

- For linear or nonlinear systems

# Examples in Matlab

- https://se.mathworks.com/help/mpc/gs/control-of-a-single-input-single-output-plant.html

Command: openExample("mpc/mpcdoubleint")

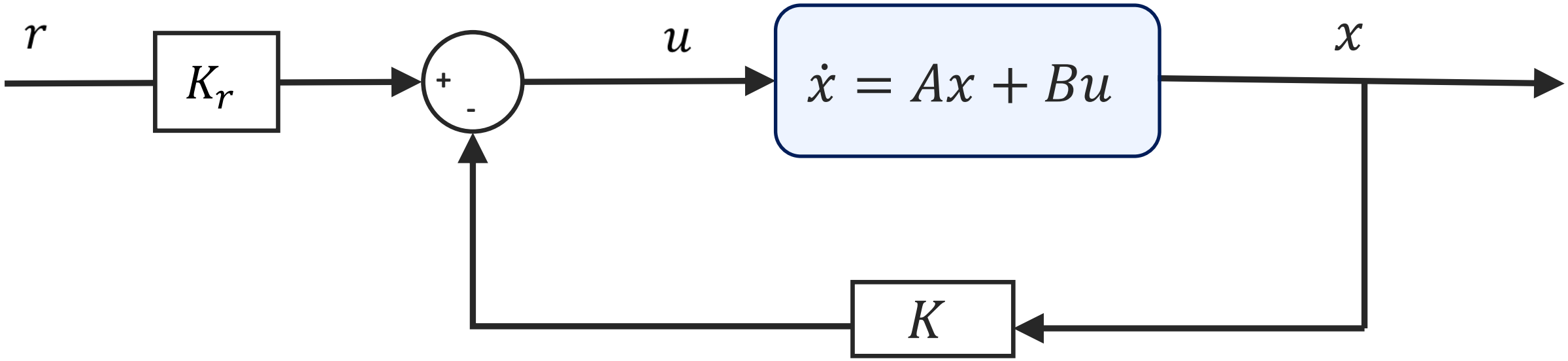
- https://se.mathworks.com/help/mpc/gs/control-of-a-multi-input-multi-output-nonlinear-plant.html

Command: openExample("mpc/mpcnonlinear")

# Model Reference Adaptive Control (MARC)

https://en.wikipedia.org/wiki/Model_predictive_control
https://laurentlessard.com/teaching/me7247/lectures/lecture%2020%20-%20model%20predictive%20control.pdf

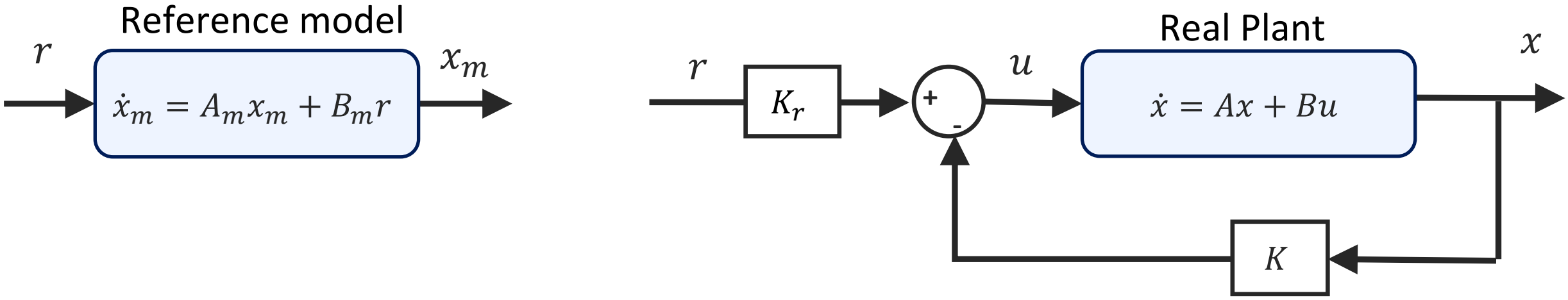
# State feedback control




$$u = -Kx + K_r r$$

# Model reference Control

We specify a reference model that we want the closed-loop system to match.



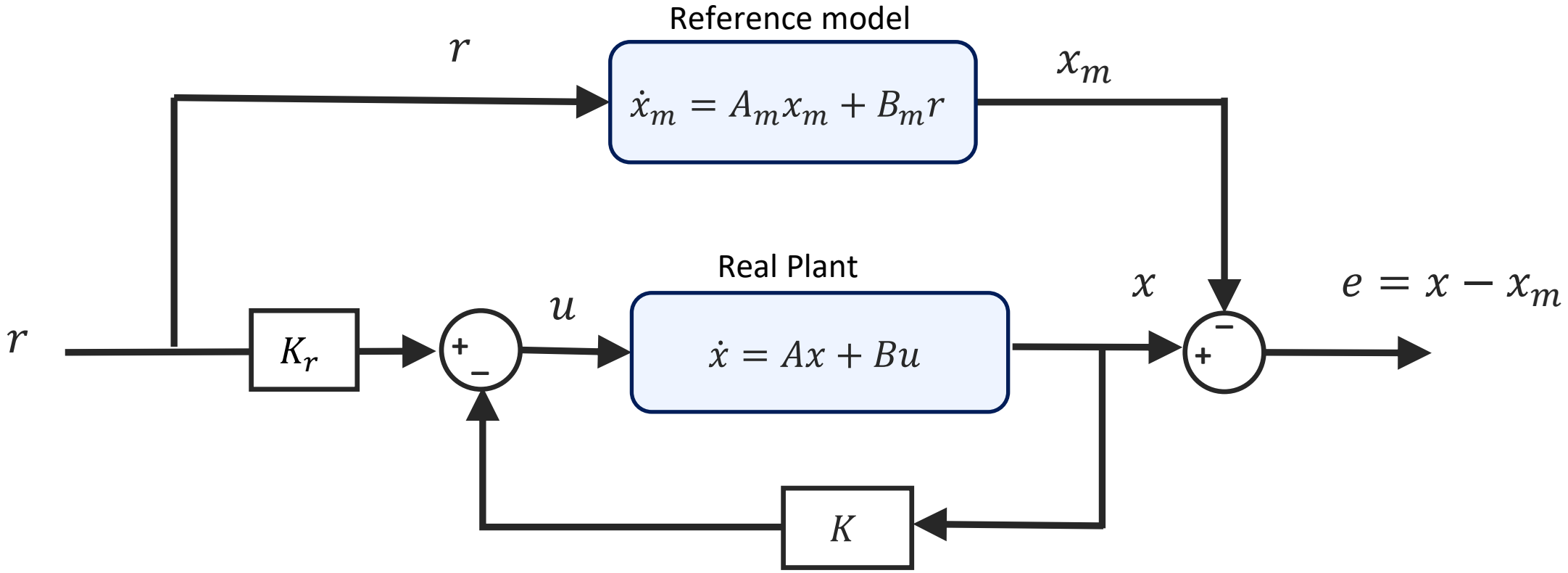

$$\dot{x}_m = A_m x_m + B_m r \quad \text{set equal} \quad \dot{x} = Ax + Bu = (A - BK)x + BK_r r$$

$$u = -Kx + K_r r$$

$$A_m = A - BK$$
$$B_m = B\ K_r$$

Model matching conditions

# Model reference control




$$\dot{x} = (A - BK)x + BK_r r$$
$$\dot{x}_m = A_m x_m + B_m r$$

Model matching conditions

$$A_m = A - BK$$
$$B_m = B\ K_r$$

# Model reference control $u = -Kx + K_r r$

$\dot{x} = Ax + Bu = (A - BK)x + BK_r r$

$\dot{x}_m = A_m x_m + B_m r$

- The closed-loop system match the reference model.
- Matching conditions:

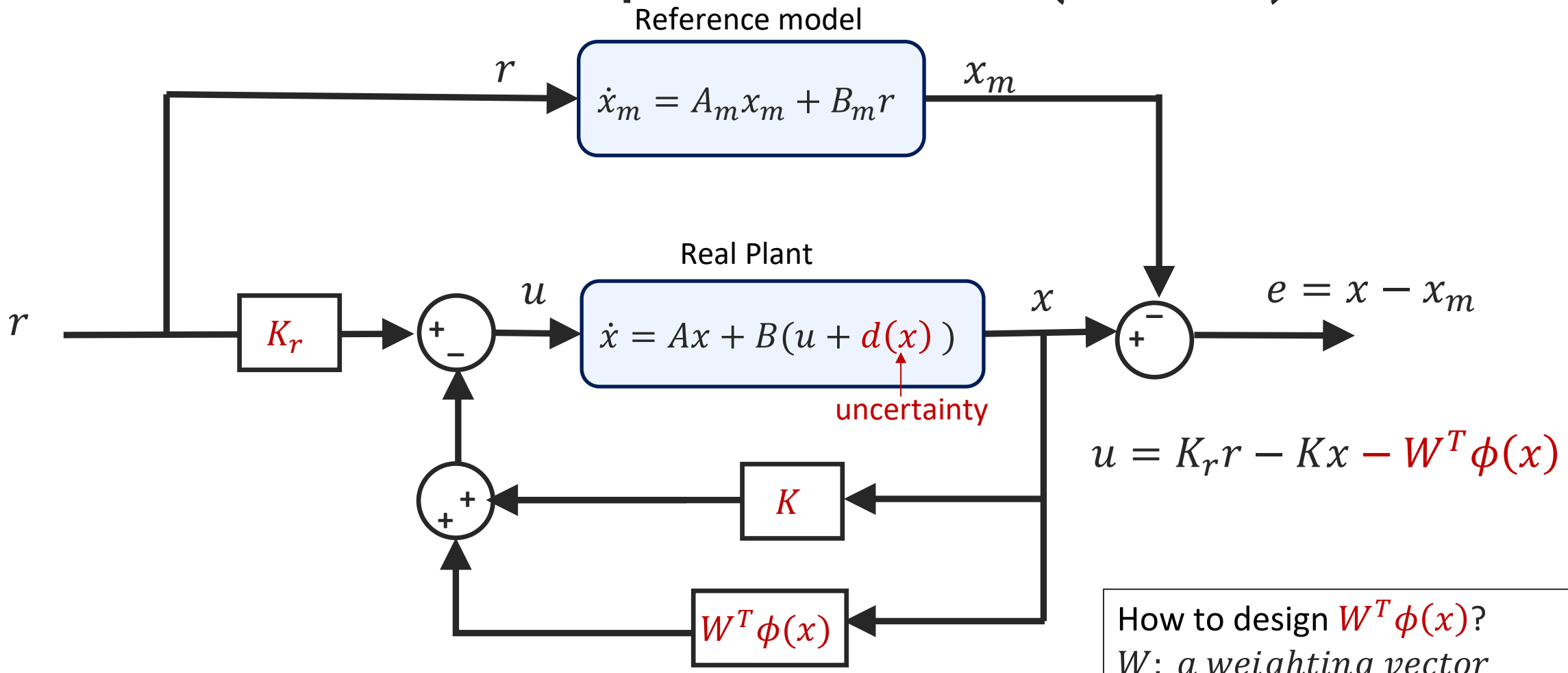
$A_m = A - BK$
$B_m = B\ K_r$

- The error

$e = x - x_m$
$\dot{e} = x\ - \dot{x}_m = (A - BK)x + BK_r r - A_m x_m - B_m r = A_m(x - x_m) = A_m e$

- The reference model $A_m$ is chosen by design to be a Hurwitz matrix (all eigenvalues have strictly negative real parts).
- It ensures the closed-loop stability, such as $\lim_{t\to\infty} e(t) = 0$.

# Model reference adaptive control (MRAC)




If $W^T\phi(x) = d(x)$, we achieve the system

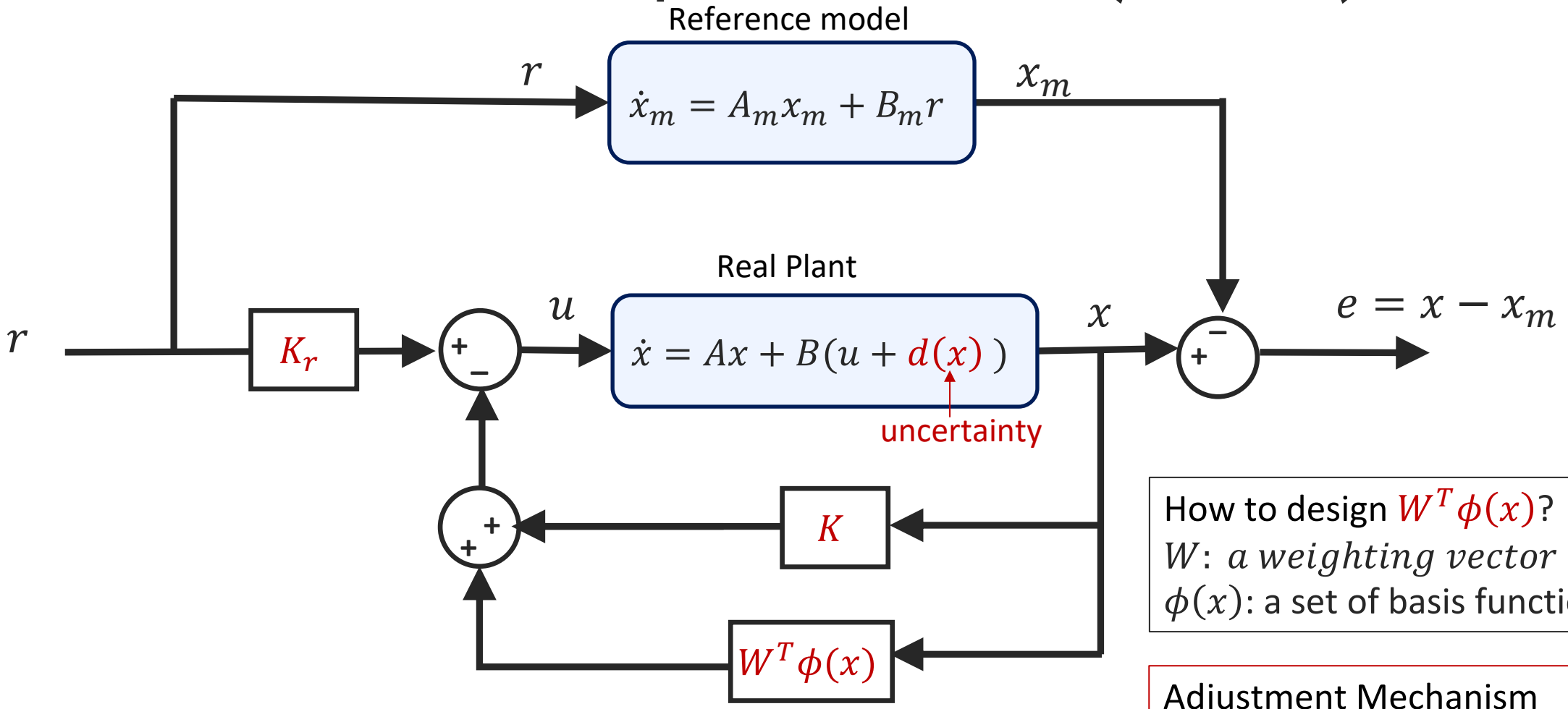
$\dot{x} = Ax + B(K_r r - Kx - W^T\phi(x) + d(x)) = (A - BK)x + BK_r r$

How to design $W^T\phi(x)$?

$W$: *a weighting vector*

$\phi(x)$: a set of basis functions

# Model reference adaptive control (MRAC)




How to design $W^T\phi(x)$?
$W$: *a weighting vector*
$\phi(x)$: a set of basis functions

Adjustment Mechanism
- Learn the optimal $W$
- Can Learn $K, K_r$ *too*

$$u = K_r r - Kx - W^T\phi(x)$$

# Model reference adaptive control (MRAC)

Control: $u = K_r r - Kx - W^T \phi(x)$

$\dot{x} = Ax + B\big(u + d(x)\big) = Ax + B(K_r r - Kx - W^T \phi(x) + d(x))$

$= (A - BK)x + BK_r r + B(d(x) - W^T \phi(x))$

$\dot{x}_m = A_m x_m + B_m r$

$Set \;\; A_m = A - BK \;, B_m = B\; K_r$

$e = x - x_m$

$\dot{e} = x \; - \dot{x}_m = A_m e + B\big(d(x) - W^T \phi(x)\big)$

If $we\ know\ d(t) = W^T \phi(x)$, $\dot{e} = x \; - \dot{x}_m = A_m e$.

Since the reference model is a stable system (eigenvalues of A =poles of reference model are negative), the final controlled system is stable.

# Model reference adaptive control (MRAC)

If $d(x) = W^T\phi(x)$ is unknown, design $\widehat{W}$ is the estimate of weight vector $W$

$$u = K_r r - Kx - \widehat{W}^T\phi(x)$$

$$\dot{e} = x - \dot{x}_m = A_m e + B\left(W^T\phi(x) - \widehat{W}^T\phi(x)\right) = A_m e + B\widetilde{W}^T\phi(x)$$

where $\widetilde{W} = W - \widehat{W}$ is the estimation error. Using LYP method for single input system:

$$V = \frac{1}{2}e^T Pe + \frac{1}{2}\Gamma^{-1}\widetilde{W}^T\widetilde{W}\ ,\ \text{where}\ A_m^T P + PA_m = -Q, P = P^T > 0, Q = Q^T > 0, \Gamma > 0$$

$$\dot{V} = \frac{1}{2}\dot{e}^T Pe + \frac{1}{2}e^T P\dot{e} + \Gamma^{-1}\widetilde{W}^T\dot{\widetilde{W}}$$

$$= \frac{1}{2}e^T(A_m^T P + PA_m)e + \widetilde{W}^T\phi(x)e^T PB - \Gamma^{-1}\widetilde{W}^T\dot{\widehat{W}}$$

$$= -\frac{1}{2}e^T Qe - \Gamma^{-1}\widetilde{W}^T\left(\dot{\widehat{W}} - \Gamma\,\phi(x)e^T PB\right)$$

$$\dot{\widehat{W}} = \Gamma\,\phi(x)e^T PB$$

$$\dot{V} = -\frac{1}{2}e^T Qe \le 0$$

- Since $V$ is bounded and $\dot{V}$ is uniformly continuous, then $\lim_{t\to\infty} e(t) = 0$

# Model reference adaptive control (MRAC)

If $A$ is unknown, design $\widehat{W}$ is the estimate of weight vector $W$

$$u = K_r r - Kx - \widehat{W}^T \phi(x)$$

$$\dot{e} = x - \dot{x}_m = A_m e + B\left(W^T\phi(x) - \widehat{W}^T\phi(x)\right) = A_m e + B\widetilde{W}^T\phi(x)$$

where $\widetilde{W} = W - \widehat{W}$ is the estimation error. Using LYP method for single input system:

$$V = \frac{1}{2}e^T Pe + \frac{1}{2}\Gamma^{-1}\widetilde{W}^T\widetilde{W}\ ,\ \text{where}\ A_m^T P + PA_m = -Q, P = P^T > 0, Q = Q^T > 0, \Gamma > 0$$

$$\dot{V} = \frac{1}{2}\dot{e}^T Pe + \frac{1}{2}e^T P\dot{e} + \Gamma^{-1}\widetilde{W}^T\dot{\widetilde{W}}$$

$$= \frac{1}{2}e^T(A_m^T P + PA_m)e + \widetilde{W}^T\phi(x)e^T PB - \Gamma^{-1}\widetilde{W}^T\dot{\widehat{W}}$$

$$= -\frac{1}{2}e^T Qe - \Gamma^{-1}\widetilde{W}^T\left(\dot{\widehat{W}} - \Gamma\,\phi(x)e^T PB\right)$$

$$\dot{\widehat{W}} = \Gamma\,\phi(x)e^T PB$$

$$\dot{V} = -\frac{1}{2}e^T Qe \le 0$$

- Since $V$ is bounded and $\dot{V}$ is uniformly continuous, then $\lim_{t\to\infty} e(t) = 0$

# Additional

- MRAC- example in Matlab

https://se.mathworks.com/help/slcontrol/ug/model-reference-adaptive-control-of-aircraft-undergoing-wing-rock.html?s_eid=PSM_15028

- An adaptive example, see attached PDF